

# Selective Underwater Image Enhancement for Object Detection with YOLO11

1<sup>st</sup> Nguyen Quoc Bao

*The University of Danang – University of Science and Technology*

Da Nang, Vietnam

102250412@hv.dut.udn.vn

2<sup>nd</sup> Phan Le Van Minh

*The University of Danang – University of Science and Technology*

Da Nang, Vietnam

102250417@hv.dut.udn.vn

**Abstract**—Underwater image enhancement (UIE) is often used as a pre-processing step for underwater object detection, but an enhancer that helps one image may hurt another. We study selective enhancement: for each image, a lightweight ResNet18 selector chooses between the original image and the outputs of two pretrained enhancers, SCNet and Semi-UIR, before the image is passed to a YOLO11n detector. Selector labels are produced with a three-fold out-of-fold protocol based on per-image mean average precision (mAP).

**Index Terms**—underwater image enhancement, underwater object detection, selective enhancement, YOLO11, RUOD, UIEB

## I. INTRODUCTION

Underwater images suffer from wavelength-dependent absorption and scattering, which cause color casts, low contrast and haze. Underwater image enhancement (UIE) methods such as SCNet [1] and Semi-UIR [2] restore visual quality and are frequently placed in front of downstream vision models under the assumption that a better-looking image is also easier to analyse.

Recent evidence challenges this assumption for object detection. Awad et al. [3] evaluated nine enhancement methods with modern detectors and found that traditional image quality metrics do not reliably predict detection performance, and that dataset-level conclusions hide large per-image variability: enhancement clearly helps on many low-quality inputs but degrades others. Their mixed-set upper-bound analysis shows that choosing the best version for each image could yield substantial gains over both the original and the fully enhanced datasets, but it assumes an ideal, ground-truth-aware choice.

In this work we turn that upper bound into a practical pipeline. For each image, a lightweight classifier selects one of three versions, namely the original image, the SCNet output, or the Semi-UIR output, which is then passed to a YOLO11n detector [cite: YOLO11 / Ultralytics]. Our contributions are:

- A reproducible inference pipeline for SCNet and Semi-UIR with their released weights, running at native resolution on current PyTorch, and verified against the published UIEB results (Section V-A).

- A per-image analysis on UIEB showing that the best version varies from image to image even under full-reference fidelity metrics (Section V-B).
- (*In progress.*) A leakage-free labelling protocol based on three-fold out-of-fold per-image mAP, and a ResNet18 selector evaluated on RUOD against fixed-enhancer, random and oracle baselines (Section V-C).

## II. RELATED WORK

### A. Underwater Image Enhancement

SCNet [1] targets the diversity of underwater degradations by learning *water-type desensitized* representations. At each scale of a U-Net, it whitens activations across spatial dimensions for every instance and removes channel-wise correlation by standardising and re-injecting the first two moments of the activations, so that the decoder can reconstruct the clean signal independently of the water type.

Semi-UIR [2] addresses the shortage of paired training data with a mean-teacher semi-supervised framework. A *reliable bank* stores the “best-ever” teacher outputs, ranked by a no-reference image quality metric, as pseudo ground truth, and a contrastive regularisation term reduces confirmation bias from wrong pseudo-labels. Its released restoration network takes both the image and an estimated luminance map as input.

Both methods are trained and evaluated on UIEB [4] using full-reference metrics (PSNR, SSIM), which measure fidelity to a reference image rather than usefulness for a downstream task.

### B. Enhancement for Underwater Object Detection

Awad et al. [3] propose a unified quality–detection evaluation framework that combines (1) a distribution-level composite quality index, (2) a per-image detection protocol based on COCO-style mAP, and (3) a mixed-set upper bound that quantifies the gain achievable by ideal selective enhancement. We adopt their per-image mAP protocol to build training labels, and we use their upper bound as the oracle reference for our learned selector.



Fig. 1. Qualitative example produced by our pipeline. Left to right: original image, Semi-UIR, SCNet.

### III. METHOD

#### A. Overview

Let  $x$  be an input image and let  $E_{SC}$  and  $E_{SU}$  denote the frozen SCNet and Semi-UIR enhancers. The candidate set is

$$\mathcal{V}(x) = \{v_0, v_1, v_2\} = \{x, E_{SC}(x), E_{SU}(x)\}. \quad (1)$$

A selector  $f$  predicts an index  $\hat{k} = \arg \max_k f(x)_k$  and the detector  $D$  is applied to  $v_{\hat{k}}$ . In deployment only the chosen enhancer needs to be executed.

#### B. Enhancers

Both enhancers use the authors' pretrained weights without retraining and run at the native input resolution (long side 640 for RUOD). The original repositories were forked with minimal compatibility fixes for a current PyTorch environment; all project logic lives in separate wrapper scripts.

**SCNet.** Only the enhancement network (decoder) is loaded; the VGG16 branch, which serves only the training loss, is discarded. Because the network contains three  $2\times$  down-sampling stages, the input is reflect-padded to a multiple of 8, and the output is cropped back to the input size and quantised as in the official evaluation script.

**Semi-UIR.** The luminance map required by the network is computed automatically: the image is blurred with Gaussian kernels of  $\sigma \in \{15, 60, 90\}$ , the logarithms of the blurred images are averaged, and the result is min-max normalised to  $[0, 255]$ . The image and its luminance map are reflect-padded to a multiple of 16, and the output is cropped back.

Enhanced images keep the original file names (JPEG, quality 95) and the YOLO labels are copied unchanged, so the three versions of every image share identical annotations. A sample result is shown in Fig. 1.

#### C. Out-of-Fold Selector Labels

Labels must come from a detector that has never seen the image; otherwise per-image mAP is inflated. The training set is split into three folds (seed 0). For each fold  $k$ , a YOLO11n model is trained for 30 epochs on all three versions of the two remaining folds, and its last checkpoint predicts the three versions of fold  $k$  with a confidence threshold of 0.001. Per-image mAP@0.5:0.95 is then computed with pycocotools following [3]; images without ground-truth boxes are assigned 0. The label of image  $i$  is

$$y_i = \begin{cases} 0 & \text{if } \max_k m_{i,k} - m_{i,0} < \delta, \\ \arg \max_k m_{i,k} & \text{otherwise,} \end{cases} \quad (2)$$

where  $m_{i,k}$  is the per-image mAP of version  $k$  and  $\delta = 0.01$ , so that the original image is preferred when enhancement brings no meaningful gain.

#### D. Selector

The selector is a ResNet18 [cite: ResNet, He et al., CVPR 2016] pretrained on ImageNet, with its last layer replaced by a 3-way classifier and trained with a class-weighted cross-entropy loss to counter label imbalance. We hold out 20% of the training images for validation (seed 0) and train for about 15 epochs. Since high accuracy can be obtained by always predicting the majority class, the checkpoint is selected by the *chosen-mAP*, i.e., the mean per-image mAP of the versions picked by the selector on the validation split, rather than by accuracy.

#### E. Final Detector and Evaluation

The final YOLO11n is trained for 50 epochs on all three versions of the full training set. On the test set, it predicts each version once; any selection policy (fixed, random, selector, oracle) is then evaluated by composing these stored predictions per image, without re-running the detector. The test set is used only for this final evaluation.

### IV. EXPERIMENTAL SETUP

#### A. Datasets

**UIEB** [4]. We use the standard 90-image paired test split to verify the enhancers against the numbers reported in their papers.

**RUOD** [cite: RUOD, Fu et al., Neurocomputing 2023]. RUOD contains 14,000 underwater images (9,800 train / 4,200 test) annotated with 10 object classes. All images are resized so that the long side is 640 pixels before any other processing; YOLO labels are normalised and therefore unchanged.

#### B. Implementation Details

All experiments run on Kaggle with an NVIDIA Tesla T4 GPU and PyTorch 2.11 (CUDA 12.8). Seed 0 is used for every split and shuffle. Detection uses the Ultralytics implementation of YOLO11n initialised from COCO-pretrained weights, with an input size of 640.

#### C. Metrics

**Image fidelity.** PSNR and SSIM are computed with scikit-image on full-resolution 8-bit RGB images against the UIEB reference (data range 255, SSIM over the colour channels), without any resizing; a size mismatch is treated as an error.

**Detection.** We report COCO mAP@0.5 and mAP@0.5:0.95. To summarise how much of the oracle headroom a selector recovers, we report the gap-closed ratio

$$G = \frac{\text{mAP}_{\text{sel}} - \text{mAP}_{\text{orig}}}{\text{mAP}_{\text{oracle}} - \text{mAP}_{\text{orig}}}. \quad (3)$$

**Runtime.** Wall-clock time per image on a single T4 GPU.

TABLE I  
PSNR / SSIM ON THE UIEB TEST SET (90 IMAGES), MEAN  $\pm$  STD.

| Method                        | Ours                    |                          | Reported |        |
|-------------------------------|-------------------------|--------------------------|----------|--------|
|                               | PSNR (dB)               | SSIM                     | PSNR     | SSIM   |
| Raw input                     | 17.90 $\pm$ 5.35        | 0.776 $\pm$ 0.137        | –        | –      |
| SCNet                         | 22.43 $\pm$ 4.43        | 0.879 $\pm$ 0.082        | 22.08    | 0.8625 |
| Semi-UIR                      | <b>24.21</b> $\pm$ 4.03 | <b>0.886</b> $\pm$ 0.069 | 24.59    | 0.901  |
| Per-image oracle <sup>†</sup> | 24.95                   | 0.901                    | –        | –      |

<sup>†</sup>Best of the three versions per image, chosen per metric.

TABLE II  
NUMBER OF UIEB TEST IMAGES (OUT OF 90) ON WHICH EACH VERSION OBTAINS THE BEST SCORE.

| Metric | Raw input | SCNet | Semi-UIR |
|--------|-----------|-------|----------|
| PSNR   | 6         | 24    | 60       |
| SSIM   | 10        | 29    | 51       |

## V. RESULTS

### A. Enhancer Verification on UIEB

Table I compares our measurements with the values reported in the original papers. SCNet is 0.35 dB above its reported PSNR and Semi-UIR is 0.38 dB below, with SSIM within 0.02 in both cases. These small gaps confirm that the released weights are loaded and run correctly. Plausible sources of the remaining differences are the SSIM implementation (window type and size), the JPEG re-encoding of our outputs (quality 95), and output rounding. Both enhancers raise PSNR by more than 4.5 dB over the raw input and also reduce its spread across images, with Semi-UIR being the stronger of the two on average.

### B. Per-Image Analysis

Average scores hide strong per-image variation. Table II counts, for each image, which version obtains the highest score. Semi-UIR is best for two thirds of the images under PSNR, but SCNet wins on 24 images and the raw input on 6. Semi-UIR beats SCNet on 63 of 90 images in PSNR and on 57 in SSIM. Enhancement lowers PSNR with respect to the raw input on 12 images for SCNet and 8 for Semi-UIR. An extreme case is image 703, whose raw PSNR is 40.63 dB (the reference is almost identical to the input), while SCNet and Semi-UIR drop it to 27.54 dB and 26.38 dB. Conversely, on image 713 the raw input scores 9.75 dB, SCNet 19.64 dB, and Semi-UIR 28.54 dB.

Choosing the best version per image (Table I, last row) yields 24.95 dB and 0.901 SSIM, a gain of 0.74 dB and 0.015 over always using Semi-UIR. Although fidelity to a UIEB reference is not the same as detection utility, this behaviour mirrors the per-image variability reported for detection by Awad et al. [3] and supports the selective-enhancement hypothesis.

**Runtime.** On a T4 GPU, including image I/O, SCNet processes a UIEB image in 0.19 s on average (warm-up excluded), whereas Semi-UIR needs 2.94 s, partly because of the CPU-side luminance estimation with large Gaussian kernels. This

TABLE III  
DETECTION RESULTS ON THE RUOD TEST SET (4,200 IMAGES) WITH YOLO11N.

| Input policy                   | mAP@0.5    | mAP@0.5:0.95 | ms/img     |
|--------------------------------|------------|--------------|------------|
| Baseline (trained on original) | <i>TBD</i> | <i>TBD</i>   | <i>TBD</i> |
| Original                       | <i>TBD</i> | <i>TBD</i>   | <i>TBD</i> |
| SCNet (all images)             | <i>TBD</i> | <i>TBD</i>   | <i>TBD</i> |
| Semi-UIR (all images)          | <i>TBD</i> | <i>TBD</i>   | <i>TBD</i> |
| Random choice                  | <i>TBD</i> | <i>TBD</i>   | <i>TBD</i> |
| <b>Selector (ours)</b>         | <i>TBD</i> | <i>TBD</i>   | <i>TBD</i> |
| Oracle (upper bound)           | <i>TBD</i> | <i>TBD</i>   | –          |

15 $\times$  cost difference matters for the selective pipeline, which only runs the enhancer it selects.

### C. Detection and Selection on RUOD

Pending: YOLO11n baseline, 3-fold labels, selector training. Fill Table III from results.md after the number freeze.

Table III will report detection accuracy on the 4,200 RUOD test images for every input policy, together with the label distribution of the selector training set and its validation accuracy and confusion matrix.

## VI. CONCLUSION

We presented a selective-enhancement pipeline in which a ResNet18 selector chooses, for each underwater image, between the original and the outputs of SCNet and Semi-UIR before YOLO11n detection. At the current stage, both enhancers are integrated with their released weights and reproduce the published UIEB results within 0.4 dB PSNR. A per-image analysis shows that no single version is best everywhere and that an oracle choice improves PSNR by 0.74 dB over the best fixed enhancer, which motivates learning the choice.

The next steps are to enhance the full RUOD dataset, train the YOLO11n baseline and the three fold models, generate out-of-fold selector labels, and evaluate the selector against fixed, random and oracle policies. Limitations include the use of full-reference fidelity metrics for the enhancer check, which do not measure detection utility, and the considerably higher cost of Semi-UIR. Adding a third enhancer such as ACDC is left as optional future work.

## REFERENCES

- [1] Z. Fu, X. Lin, W. Wang, Y. Huang, and X. Ding. *Underwater Image Enhancement via Learning Water Type Desensitized Representations*. Mar. 14, 2022. DOI: 10.48550/arXiv.2102.00676. arXiv: 2102.00676 [cs.CV]. URL: <http://arxiv.org/abs/2102.00676> (visited on 10/02/2026). Pre-published.
- [2] S. Huang, K. Wang, H. Liu, J. Chen, and Y. Li. *Contrastive Semi-Supervised Learning for Underwater Image Restoration via Reliable Bank*. Apr. 4, 2023. DOI: 10.48550/arXiv.2303.09101. arXiv: 2303.09101 [cs.CV]. URL: <http://arxiv.org/abs/2303.09101> (visited on 10/06/2026). Pre-published.
- [3] A. Awad, A. Saleem, S. Paheding, E. Lucas, S. Al-Ratrout, and T. C. Havens. “Revisiting Underwater Image Enhancement for Object Detection: A Unified Quality–Detection Evaluation Framework”. In: *Journal of Imaging* 12.1 (Jan. 2026), p. 18. DOI: 10.3390/jimaging12010018.
- [4] C. Li et al. *An Underwater Image Enhancement Benchmark Dataset and Beyond*. Nov. 26, 2019. DOI: 10.48550/arXiv.1901.05495. arXiv: 1901.05495 [cs.CV]. URL: <http://arxiv.org/abs/1901.05495> (visited on 10/01/2026). Pre-published.